AIORI-3 HACKATHON · CYBERSECURITY · PS-011

Atlas

Authorised Attack-Surface Inventory and Change Monitor

TEAM NAME

Gradient

TEAM CODE

A3-PS011-TC467

TEAM MEMBERS

Shreyas Mohanty

Arnav Sharath

Guide: Dr. Animesh Giri

Table of contents

01	Cover page	1
02	Table of contents	2
03	Problem description	3
04	Solution proposed	5
05	Optimization proposed by the team	7
06	Solution architecture and design	10
07	Timeline of delivery	12
08	References	13
09	Conclusion	14

Exposed assets pile up quietly over time

Organisations running multi-cloud infrastructure keep adding internet-facing assets and rarely remove them. Each forgotten asset is unmonitored, unpatched and still trusted under the organisation's name.

Key issues

- **Unmaintained test subdomains**: staging and demo names stay live after projects end.
- **Expired TLS certificates**: lapsed certificates break trust and signal neglect.
- **Open service ports**: services opened temporarily stay exposed.
- **Dangling DNS CNAME records**: names still point to decommissioned cloud instances.

Why it matters

These are exactly the assets attackers look for. A dangling record can let an outsider serve content on a trusted subdomain.

What the system must deliver

Key requirements

- Monitor up to 20 lab domains and 10 hosts.
- Check selected ports using 4 documented checks.
- Track domains, DNS records, TLS certificates and exposed services.
- Perform scheduled inventory comparisons.
- Detect seeded expired certificates, unexpected services and potential dangling DNS records.
- Report false positives.

Ground rules

- Use only written, organiser-approved target scope.
- No exploitation or unrestricted scanning.
- No claim that every dangling record is exploitable.

04 · SOLUTION PROPOSED

An inventory that checks itself on a schedule

Our proposed tool ATLAS catalogs every in-scope domain and host, re-checks them on a schedule, and raises an alert only when a finding is confirmed. It runs four documented checks:

DNS check

Collects A, AAAA, CNAME, MX, TXT and NS records for every in-scope domain.

TLS certificate check

Reads each certificate's expiry date, issuer and covered names.

Port and service check

Probes the documented ports and records which service answers.

Dangling DNS check

Follows each CNAME and checks whether its target still exists.

04 · SOLUTION PROPOSED

How each scan round works

Step 1

Scope gate

Every probe is checked against the approved list.

Step 2

Collect

The four checks run in parallel and save facts.

Step 3

Compare

Diff against the baseline and apply always-on rules.

Step 4

Confirm

Recheck suspicious findings before alerting.

Step 5

Alert and report

Send alerts with severity and evidence.

Two detection layers

Always-on rules catch issues present from the first scan; change detection catches what is new since the baseline.

Split stack

A C++ engine collects facts; a Python layer stores, compares, alerts and reports.

05 · OPTIMIZATION PROPOSED BY THE TEAM

Six optimizations and enhancements

01 Policy-aware baseline

Always-on rules plus change detection, so issues present on day one are still caught.

02 Confirm before alert

Recheck findings and query several DNS resolvers to filter out network flukes.

03 Confidence-scored dangling DNS

Combine NXDOMAIN, provider fingerprints and resolver agreement into high, medium or low candidates.

04 TTL- and expiry-aware scheduling

Recheck records when their TTL allows change; check certificates more often near expiry.

05 Enforced, audited scope

Every probe passes a scope gate and is logged; out-of-scope discoveries are reported, never scanned.

06 Alert grouping

Related findings merge into one incident with severity and evidence, cutting alert fatigue.

05 · OPTIMIZATION PROPOSED BY THE TEAM

Before and after comparison

Area	Typical approach (before)	Our approach (after)
Detection logic	Alert only on changes since last scan	Change detection plus always-on rules
Noise control	Alert on first observation	Rechecks and multi-resolver confirmation
Dangling DNS	Yes or no flag	Scored candidates with evidence
Scan timing	Fixed interval, rescan everything	TTL- and expiry-aware rescans
Scope safety	Trusted to the operator	Enforced in code, every probe logged
Alerting	One alert per raw finding	Grouped incidents with severity
Evaluation	Rarely measured	Precision, recall and FP rate measured

05 · OPTIMIZATION PROPOSED BY THE TEAM

Optimization process for every finding

Finding

A rule violation or a change from the baseline.

Recheck

Probe again twice, a few seconds apart.

Cross-verify

Ask several DNS resolvers and compare their answers.

Score

Assign severity and confidence from all signals.

If confirmed

Alert raised

One grouped incident with evidence.

If not confirmed

Suppressed, logged

Counted in the false-positive analysis.

Two layers joined by one JSON contract

C++ scanning engine

Collects facts safely

- Scope gate and probe audit log
- DNS checks with c-ares
- TLS checks with OpenSSL
- Port checks with Asio
- Dangling DNS checks with libcurl
- Rechecks for confirmation

→

</>

JSON snapshot

Flat facts, errors and probe count for each round.

→

Python monitor layer

Remembers, compares and alerts

- Scheduler with APScheduler
- Snapshot store in SQLite
- Diff and always-on rules
- Scoring and alert grouping
- Webhook and email alerts
- Dashboard and reports (FastAPI)

Testbed: a Docker lab with a seeded expired certificate, an unexpected service and a dangling CNAME.

End-to-end flow of one scan round

Scheduler
Starts each round; rescans by TTL and expiry.

Scope gate
Only approved targets pass; every probe is logged.

Four checks
DNS, TLS, ports and dangling DNS run in parallel.

Snapshot store
Facts saved with timestamps in SQLite.

Compare
Diff against the baseline plus always-on rules.

Confirm
Rechecks and multi-resolver agreement.

Alert and report
Grouped alerts, change reports and the false-positive log.

07 · TIMELINE OF DELIVERY

Milestones and deadlines

Week 1

Foundations

JSON contract, repository setup and a Docker lab with seeded issues.

Week 2

Core checks

C++ port and DNS checks; Python store, diff and rules.

Week 3

Full coverage

TLS and dangling checks; confirmation, scoring and alerts.

Week 4

Integration

End-to-end runs, dashboard and change reports.

Week 5

Evaluation and delivery

False-positive analysis, final report and demonstration.

Tools, resources and references

1. AIORI-3 Hackathon, PS-011: Authorised Attack-Surface Inventory and Change Monitor (problem statement and proposal structure).
2. OWASP Foundation, Web Security Testing Guide, owasp.org/www-project-web-security-testing-guide.
3. M. Husák and L. Sadlek, "Attack Surface Management: State of the Art and Operational Challenges," IEEE NetSoft 2025, doi:10.1109/NETSOFT64993.2025.11080588.
4. A. Allahbakhsh, "Automated External Attack Surface Mapping: Developing a High-Performance Reconnaissance Engine using Concurrent Systems in Go," Chalmers University of Technology, 2026.
5. M. C. Rossi, "Enhancing cyber assets visibility for effective attack surface management," M.Sc. thesis, University of Turku, 2023.
6. EdOverflow, "can-i-take-over-xyz": subdomain takeover fingerprints, github.com/EdOverflow/can-i-take-over-xyz.

Engine (C++)

CMake, c-ares, OpenSSL, Asio, libcurl

Monitor (Python)

SQLite, APScheduler, FastAPI

Lab and validation

Docker Compose, CoreDNS, nginx, Redis, Nmap

09 · CONCLUSION

Know every exposed asset, and trust every alert

Atlas gives a defined organisation a living inventory of its domains, DNS records, TLS certificates and exposed services. It runs four documented checks on a schedule, detects seeded issues, and measures its own false positives.

Significance

Targets the gaps recent work names: persistent tracking, timely scans and noisy findings, all within an authorised scope.

Next steps

- Attend mentoring sessions and clear doubts.
- Build the model following proper guidance.
- Run the evaluation and prepare a working model.